PROJECT DOCUMENTATION

AI-Powered RAG Chatbot for Course Query Assistance

Built using n8n, Google Gemini Embeddings, Pinecone and OpenAI GPT-5 Mini

Submitted by

Pentakota Suresh Appalaraju

Data Science and GenAI

Electrical and Electronics Engineering

Email: psappalaraju888@gmail.com

July 2026

Table of Contents

Table of Contents 2
1. Abstract 3
2. Introduction 3
3. Problem Statement 3
4. Objectives 3
5. System Architecture 3
5.1 Indexing Pipeline (Offline / Manual) 4
5.2 Query Pipeline (Real-time / Conversational) 4
6. Technology Stack 5
7. Node-Level Documentation 5
8. Data Flow Summary 6
9. Advantages 6
10. Applications 6
11. Limitations 6
12. Future Enhancements 6
13. Conclusion 7
14. References 7

1. Abstract

This project implements a Retrieval-Augmented Generation (RAG) chatbot that answers user questions about course offerings using an internal knowledge base. The system is built entirely on the n8n workflow automation platform and combines document ingestion, vector embeddings, semantic search and a large language model into a single automated pipeline. Course information stored as a PDF on Google Drive is extracted, split into manageable chunks, converted into vector embeddings using Google Gemini, and stored in a Pinecone vector database. When a user sends a question through the chat interface, an AI agent powered by OpenAI's GPT-5 Mini model retrieves the most relevant chunks from Pinecone and generates a grounded, context-aware answer. The result is an assistant that can accurately respond to queries about courses, syllabus and training details without hallucinating information that is not present in the source document.

2. Introduction

Organisations that offer training programs, such as Naresh IT, often maintain course catalogues, syllabi and FAQs in static documents like PDFs. Prospective students typically have to read through lengthy files or contact support staff to get answers to simple questions such as "What topics are covered in the Data Science course?" or "What is the duration of the Java course?". This project automates that process by building an AI chatbot that can read the source PDF once, understand its contents semantically, and then answer natural-language questions instantly and accurately.

The solution follows the Retrieval-Augmented Generation (RAG) pattern, which is the industry-standard approach for grounding a language model's responses in a specific, private knowledge base rather than relying purely on the model's general training data. Instead of writing custom backend code, the entire pipeline is implemented visually using n8n, a low-code workflow automation tool, which makes the system easy to understand, modify and maintain.

3. Problem Statement

Manually searching through PDF documents for specific information is slow and inconvenient, and a plain large language model cannot answer questions about private or recently updated course content because it has no knowledge of that document. There is a need for a system that can:

- Automatically ingest and understand the contents of a course PDF
- Store that knowledge in a searchable format
- Allow users to ask questions in plain English through a chat interface
- Return accurate answers that are grounded strictly in the source document, and clearly state when information is not available

4. Objectives

1. Automate extraction of course information from a Google Drive PDF.
2. Split and embed the extracted text so it can be searched semantically.
3. Store the embeddings in a scalable, cloud-hosted vector database (Pinecone).
4. Build a conversational AI agent that retrieves relevant knowledge and answers user questions.
5. Deliver the assistant through a simple, public chat interface that requires no custom front-end development.

5. System Architecture

The workflow is divided into two independent pipelines that both operate on the same Pinecone index, n8n-index1-rag. The Indexing Pipeline is run manually whenever the source document needs to be (re)loaded into the vector store. The Query Pipeline runs continuously in the background and is triggered every time a user sends a message through the public chat widget. The complete architecture is shown below.

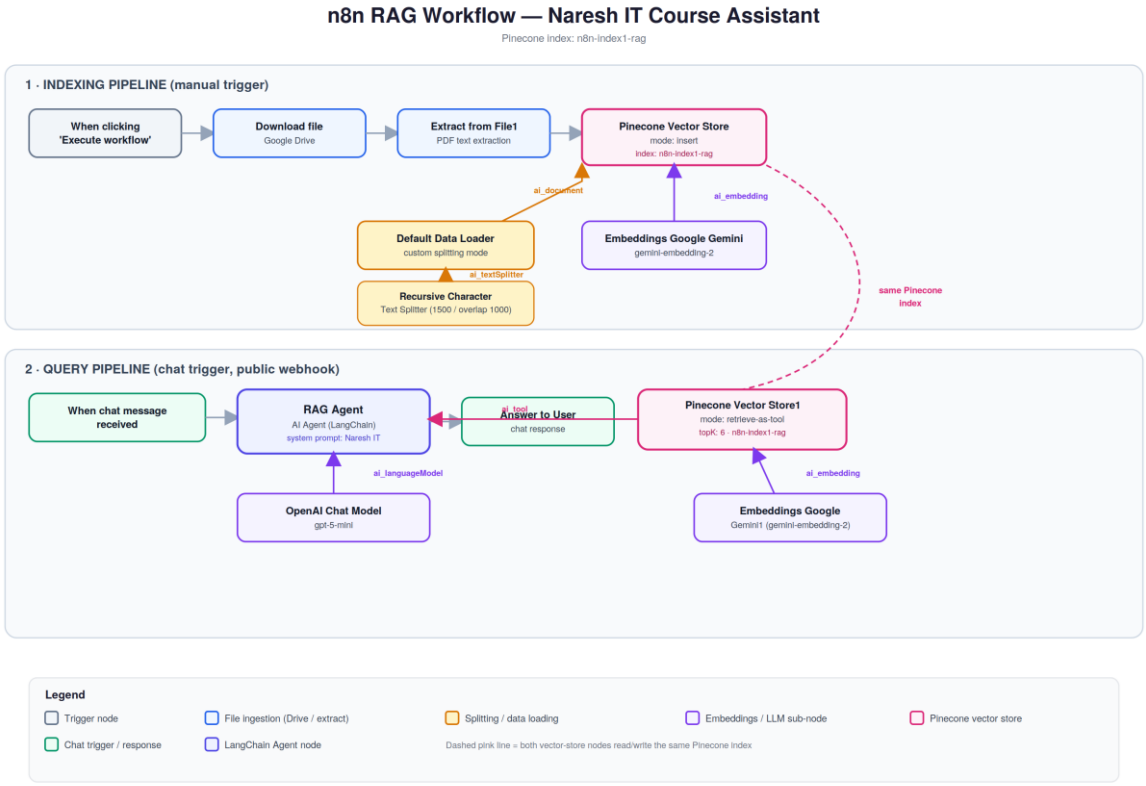


Figure 1: End-to-end n8n RAG workflow architecture

5.1 Indexing Pipeline (Offline / Manual)

This pipeline is executed on demand by clicking "Execute workflow" in n8n. It performs the one-time (or periodic) job of loading the knowledge base:

1. Manual Trigger — starts the workflow.
2. Download file — the Google Drive node downloads the target PDF (Naresh_IT_Courses) using its file ID.
3. Extract from File1 — extracts raw text content from the downloaded PDF.
4. Recursive Character Text Splitter — divides the extracted text into overlapping chunks (chunk size 1500 characters, overlap 1000 characters) so that context is preserved across chunk boundaries.
5. Default Data Loader — wraps each chunk into a LangChain document object ready for embedding.
6. Embeddings Google Gemini — converts each document chunk into a numerical vector using the gemini-embedding-2 model.
7. Pinecone Vector Store (insert mode) — writes the resulting vectors into the n8n-index1-rag Pinecone index for later retrieval.

5.2 Query Pipeline (Real-time / Conversational)

This pipeline runs whenever an end user interacts with the chatbot:

1. When chat message received — a public Chat Trigger webhook captures the user's question.

2. RAG Agent — a LangChain AI Agent node receives the question and decides how to respond, following a system prompt instructing it to always search the knowledge base before answering, and to say so if the answer cannot be found.
3. OpenAI Chat Model (gpt-5-mini) — supplies the language understanding and generation capability that powers the RAG Agent's reasoning.
4. Pinecone Vector Store1 (retrieve-as-tool mode) — exposed to the agent as a callable tool; when invoked, it embeds the query (via Embeddings Google Gemini1) and performs a similarity search against the n8n-index1-rag index, returning the top 6 most relevant chunks.
5. The RAG Agent combines the retrieved context with the user's question and generates a final, grounded answer, which is returned to the user through the chat interface.

6. Technology Stack

Component	Technology Used
Workflow Automation	n8n (self-hosted / cloud workflow engine)
Document Source	Google Drive (PDF file storage)
Text Processing	n8n Extract From File node + Recursive Character Text Splitter
Embedding Model	Google Gemini – gemini-embedding-2
Vector Database	Pinecone (index name: n8n-index1-rag)
Conversational LLM	OpenAI GPT-5 Mini
Agent Framework	LangChain Agent (via @n8n/n8n-nodes-langchain)
User Interface	n8n Chat Trigger (public webhook-based chat widget)

7. Node-Level Documentation

The table below documents every node used in the n8n workflow, its underlying node type, its functional role in the pipeline, and its key configuration values, as defined in the exported workflow JSON.

Node Name	n8n Node Type	Role	Key Configuration
When clicking 'Execute workflow'	Manual Trigger	Starts the indexing pipeline manually	No parameters
Download file	Google Drive node	Downloads the source PDF from Drive	File ID: Naresh_IT_Courses
Extract from File1	Extract From File	Extracts raw text from the PDF	Operation: pdf
Recursive Character Text Splitter	Text Splitter (LangChain)	Breaks extracted text into chunks	Chunk size 1500, overlap 1000
Default Data Loader	Document Default Data Loader	Wraps split text as LangChain documents	Mode: custom
Embeddings Google Gemini	Embeddings – Google Gemini	Converts text chunks into vector embeddings	Model: gemini-embedding-2
Pinecone Vector Store	Vector Store – Pinecone	Inserts embeddings into the Pinecone index	Mode: insert, Index: n8n-index1-rag
When chat message received	Chat Trigger	Public webhook that receives user questions	Public: true

Node Name	n8n Node Type	Role	Key Configuration
RAG Agent	AI Agent (LangChain)	Orchestrates reasoning and tool calls	Custom system prompt for Naresh IT
OpenAI Chat Model	LM Chat – OpenAI	Language model used by the RAG Agent	Model: gpt-5-mini
Pinecone Vector Store1	Vector Store – Pinecone (tool)	Retrieves relevant chunks for the agent	Mode: retrieve-as-tool, topK: 6
Embeddings Google Gemini1	Embeddings – Google Gemini	Embeds the incoming query for search	Model: gemini-embedding-2

8. Data Flow Summary

At indexing time, data flows in one direction: PDF → extracted text → text chunks → embeddings → Pinecone index. At query time, data flows in a loop: the user's question is sent to the RAG Agent, which calls the Pinecone retrieval tool; the tool embeds the question, searches the index, and returns matching chunks back to the agent; the agent then uses the OpenAI language model to compose a final answer from those chunks before returning it to the user. Because both pipelines write to and read from the same Pinecone index (n8n-index1-rag), any document re-indexed through the Indexing Pipeline immediately becomes available to the Query Pipeline without any additional configuration.

9. Advantages

- No-code / low-code implementation — the entire pipeline is built visually in n8n with no custom backend required.
- Answers are grounded in the actual course document, reducing hallucination compared to a plain LLM chatbot.
- Easy to update — re-running the Indexing Pipeline refreshes the knowledge base whenever the source PDF changes.
- Scalable vector search via Pinecone, suitable for much larger documents or multiple documents.
- Modular design — the embedding model, LLM, or vector database can each be swapped independently.

10. Applications

- Automated course/FAQ assistant for training institutes such as Naresh IT.
- Internal knowledge-base chatbots for HR policies, product manuals or SOPs.
- Customer support bots grounded in help-center documentation.
- Educational Q&A tools built from lecture notes or syllabi.

11. Limitations

- The chatbot's knowledge is limited strictly to what has been indexed from the source PDF.
- Large chunk overlap (1000 of 1500 characters) increases storage and embedding cost, and may need tuning for larger documents.
- The Chat Trigger is public, so access control and rate-limiting should be considered before production deployment.

12. Future Enhancements

- Support multiple source documents and automatic re-indexing on file update (via a Google Drive trigger instead of a manual trigger).
- Add authentication to the chat widget for restricted or paid course content.
- Introduce a feedback loop so incorrect answers can be flagged and the knowledge base corrected.
- Add analytics to track the most frequently asked questions.

13. Conclusion

This project demonstrates a complete, working Retrieval-Augmented Generation system built using low-code automation. By combining Google Drive, Google Gemini embeddings, Pinecone vector search and OpenAI's GPT-5 Mini model within n8n, the workflow is able to turn a static PDF into an interactive, accurate, and always-available chatbot. The architecture is intentionally modular, making it straightforward to extend to other documents, other embedding or language models, or additional business use cases beyond course information.

14. References

- n8n Documentation — https://docs.n8n.io
- Pinecone Documentation — https://docs.pinecone.io
- Google Gemini API Documentation — https://ai.google.dev
- OpenAI API Documentation — https://platform.openai.com/docs
- LangChain Documentation — https://js.langchain.com